Regression

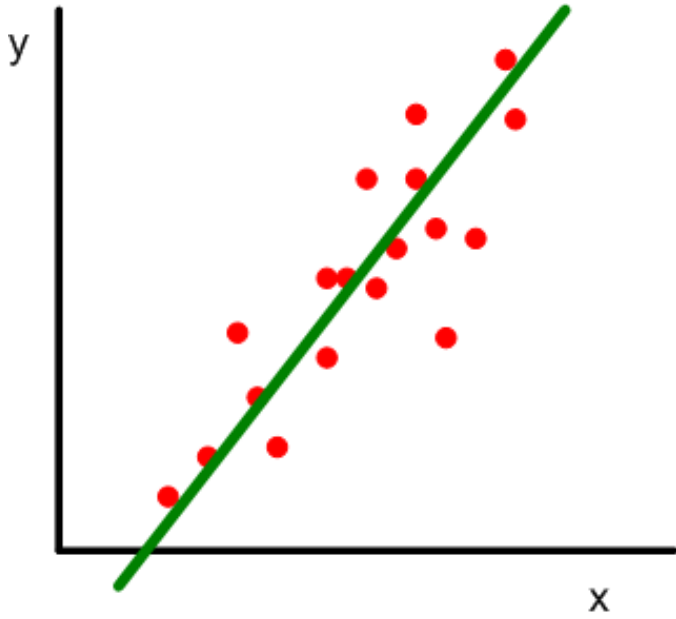
Machine learning

Ing. Tomáš Vičar, Ph.D.
Department of Biomedical Engineering,
FEEC, Brno University of Technology

Introduction

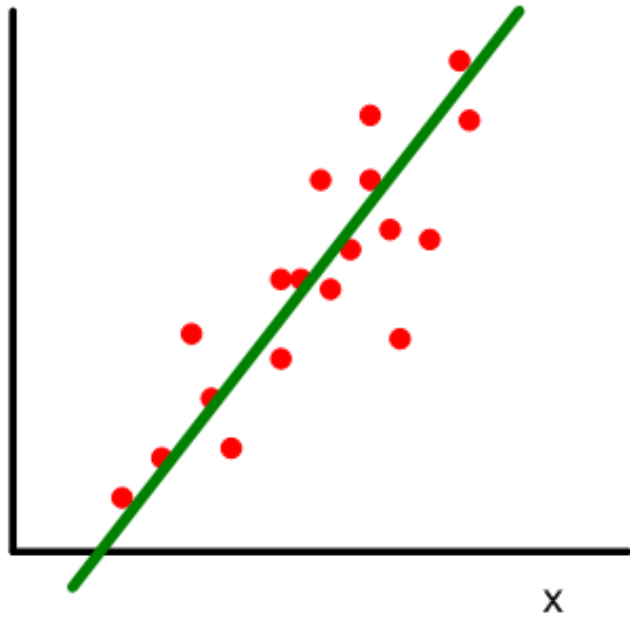
Regression is an area of statistical modeling.

It estimates the relationships between a **dependent variable** (outcome, response variable) and one or more **independent variables** (predictors, covariates, explanatory variables, features).



Introduction

Linear Regression is a **supervised** machine learning algorithm where the predicted output is continuous and has a constant slope. It's used to predict values within a continuous range, (e.g. price) rather than trying to classify them into categories (e.g. cat, dog). There are two main types:



- **univariate** regression: $y = k\,x + q$
- **multivariate** regression, e.g. $y = w_1x_1 + w_2x_2 + w_3x_3 + b$

The regression is a part of data analysis, where most univariate analysis emphasizes description of our data, while multivariate methods emphasize hypothesis testing and explanation (including feature analysis).

Linear regression

Linear regression

Data regression problems comes in the form of a (training) dataset of N observation pairs:

$$\{[\mathbf{x'}_1, y_1], [\mathbf{x'}_2, y_2], \dots [\mathbf{x'}_n, y_n]\}$$

where $\mathbf{x'}_i$ denotes the input variables and y_i denotes the output. Both variables are generally continuous.

The simplest case is, if input variable is a scalar - in that case we are solving 2-dimensional problem, i.e. fitting a line in 2D space through scattered data. In general, however, each input $\mathbf{x'}_i$ may be a collum vector of length d:

$$\mathbf{x}'_i = \left[x_{i,1}\ x_{i,2} \dots x_{i,d}\right]^T$$

d is a dimension of input vector, number of features

n is a number of datapoints

And all samples can be organized to feature matrix and label vector:

$$\mathbf{X}' = \begin{bmatrix} x_{11} & \cdots & x_{1d} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{nd} \end{bmatrix} \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

Linear regression

In this case the linear regression problem is fitting a hyperplane to a scatter of points in d+1 dimensional space.

In the case of scalar input, fitting a line to the data requires that we determine a slope *w* and a bias *b* of the regression line so that the approximate linear relationship holds between input/output data:

$$y_i \approx x_i w + b, \qquad i = 1, 2, \ldots, n$$

Linear regression

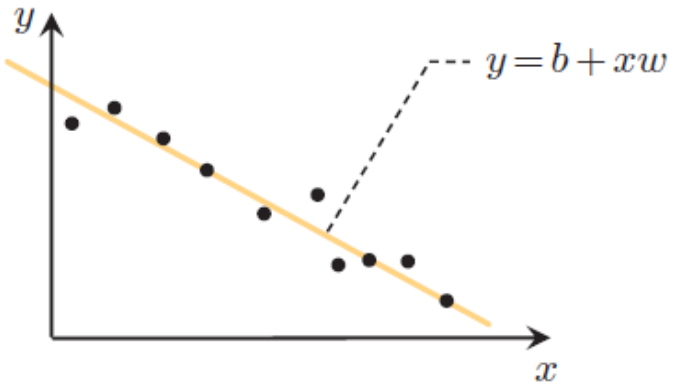
More generally, when the input dimension is d ≥ 1, we have a bias and d associated weights:

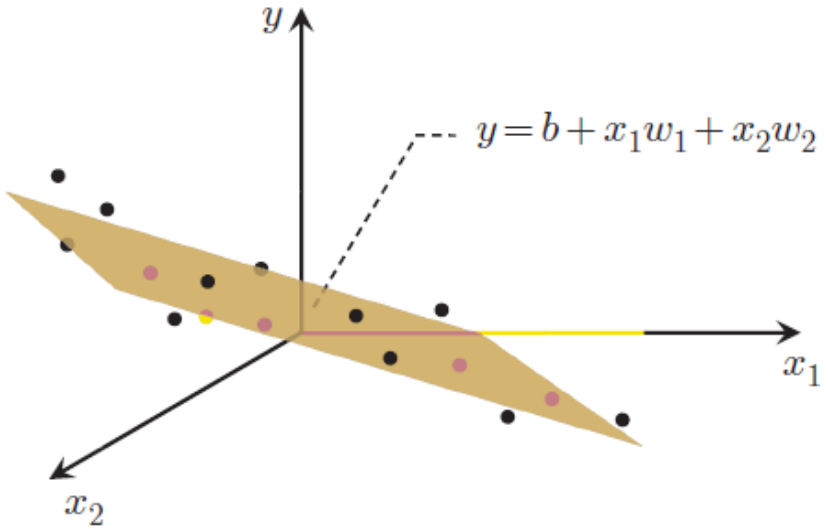
The weights determine the normal vector of the hyperplane:

$$\mathbf{w}' = [w_1\ w_2\ \ldots w_d]^T$$

The sample can be written as a vector:

$$\mathbf{x}_i' = [x_1\ x_2\ \ldots x_d]^T$$





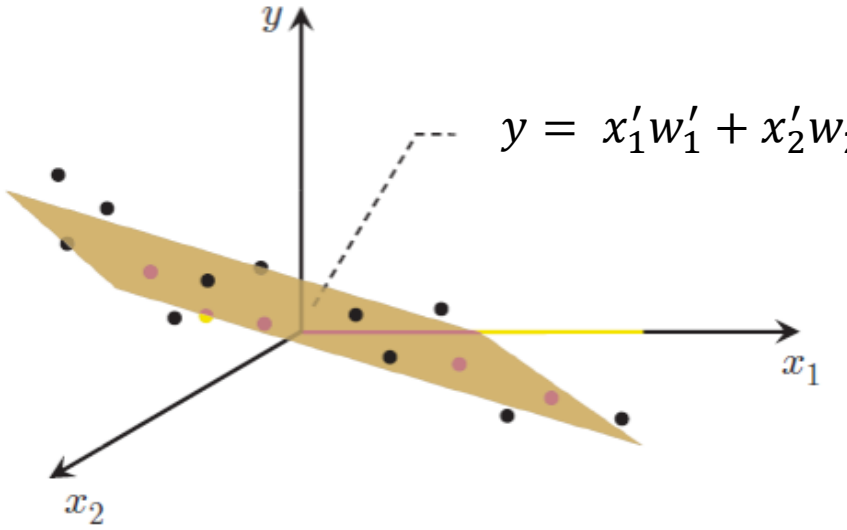
Linear regression

$$\mathbf{w} = \begin{bmatrix} b \\ \mathbf{w}' \end{bmatrix}$$

For simplification of equations, we can define:

$$\mathbf{x_i} = \begin{bmatrix} 1 \\ \mathbf{x}'_\mathbf{i} \end{bmatrix}$$

These “new” vectors are called **augmented** vectors.



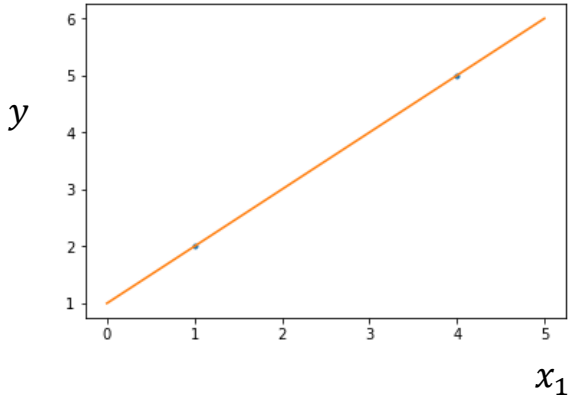
$$y = x'_1 w'_1 + x'_2 w'_2 = 1w_0 + x_1 w_1 + x_2 w_2 = \mathbf{w^T x}$$

And “new” feature matrix:

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} & \cdots & x_{1d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \cdots & x_{nd} \end{bmatrix}$$

It allows to use linear equations to fit affine functions.

Linear regression



Let assume simple case with just two 2D points:

$$\mathbf{x_1} = \begin{bmatrix} 1 \\ x_{11} \end{bmatrix} \quad \mathbf{x_2} = \begin{bmatrix} 1 \\ x_{21} \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} \\ 1 & x_{21} \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$$

Points need to satisfy line equation „*y = kx + q*“, thus we need to find weights which satisfy:

$$y_1 = x_{11}w_1 + w_0$$
$$y_2 = x_{21}w_1 + w_0$$

…two equations and two unknowns…we can solve for weights.

Matrix form:

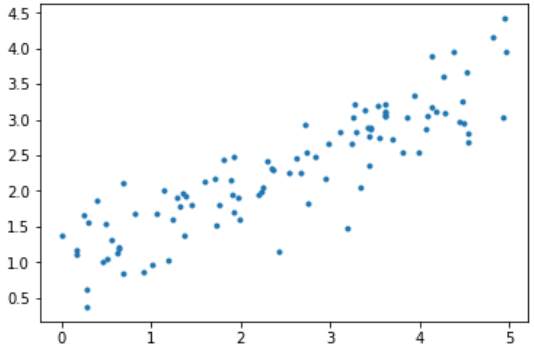
w than can be found as solution of $\mathbf{y} = \mathbf{Xw}$

$$\mathbf{w} = \mathbf{X^{-1}y}$$

For prediction, we can use $\mathbf{y_{new}} = \mathbf{X_{new}w}$ where $\mathbf{X_{new}}$ is feature matrix of new samples.

($y_{new} = \mathbf{w^T x_{new}}$ for one sample)

Linear regression



But what in case of more samples, where $\mathbf{y} = \mathbf{Xw}$
can‘t be satisfied for all points (line can‘t go through all points)?

$\mathbf{y}$ can‘t be equal to $\mathbf{Xw}$, but we can make it similar:

$$\underset{\boldsymbol{w}}{minimize}\ g(\mathbf{w}) = \sum_{i=1}^{p} (\mathbf{x_i^T w} - y_i)^2 = ||\mathbf{Xw} - \mathbf{y}||^2$$

We can use some iterative solver (e.g. gradient descent), but there is simple closed form solution.

The derivative of this cost function with respect to **w** is :

$$\nabla g(\mathbf{w}) = \sum_{i=1}^{p} 2(\mathbf{w}^T \mathbf{x_i} - y_i)\mathbf{x_i} = 2\mathbf{X^T}(\mathbf{Xw} - \mathbf{y})$$

Here, we can set the gradient to zero:

$$2\mathbf{X^T}(\mathbf{Xw} - \mathbf{y}) = 0$$

We are looking for **w**:

$$\mathbf{X^T X w} = \mathbf{X^T y}$$

And finally:

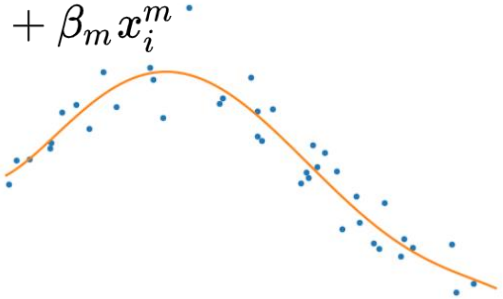
$$\mathbf{w} = (\mathbf{X^T X})^{-1}\mathbf{X^T y} = \mathbf{X}^{\dagger}\mathbf{y}$$

This is a direct solution, which needs the inverse matrix of $\mathbf{X^T X}$, which is square and often nonsingular (one that has inverse matrix). Term $\mathbf{(X^T X)^{-1} X^T}$ is also known as pseudo-inverse.

Polynomial regression

We can use same approach as for linear regression to fit different (non-linear) function.

e.g. m-th degree polynomial: $y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_m x_i^m$



Which is equivalent to transforming features and then using linear regression:

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^m \\ 1 & x_2 & x_2^2 & \dots & x_2^m \\ 1 & x_3 & x_3^2 & \dots & x_3^m \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^m \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_m \end{bmatrix},$$

Estimation of regression coefficient is formally the same as for linear regression.
Only the **X** matrix is computed beforehand.

Feature transformation

We can define feature transformation to higher dimensional space D: $\Phi: R^d \rightarrow R^D$

And replace samples in feature matrix with transformed features: $\boldsymbol{X} = \begin{bmatrix} \Phi(\mathbf{x_1})^T \\ ... \\ \Phi(\mathbf{x_n})^T \end{bmatrix}$

For example, second order polynomial feature transformation of 2D vector is

$$\Phi([x_1, x_2]^T) = [x_1^2, x_2^2, x_1 x_2, x_1, x_2, 1]^T$$

With those transformations, we can easily fit polynomials and some other nonlinear functions.

This is general way how to make (linear) model nonlinear and more powerful – able to fit more complex relations, but more prone to overfitting (can be overcame with regularization).

It also largely increases componential complexity, which can be overcame with kernel trick.